APPSEC-03: Code-Level Vulnerability Analytics

> **Series:** APPSEC — Application Security | **Notebook:** 3 of 10 | **Created:** June 2026 | **Last Updated:** 10/02/2026

Overview

Code-Level Vulnerability Analytics is the first-party counterpart to APPSEC-02's third-party (library) scope. Where RVA's library tier asks "which dependencies have known CVEs?", the code-level tier asks "which patterns in your own application code are vulnerable to known attack classes?"

This is the pillar where AppDev teams own remediation directly — there's no upstream library to upgrade, just code to change. Coverage depends on the runtime — code-level detection is documented for Java, .NET, and Go only — and on the OneAgent code module being attached.

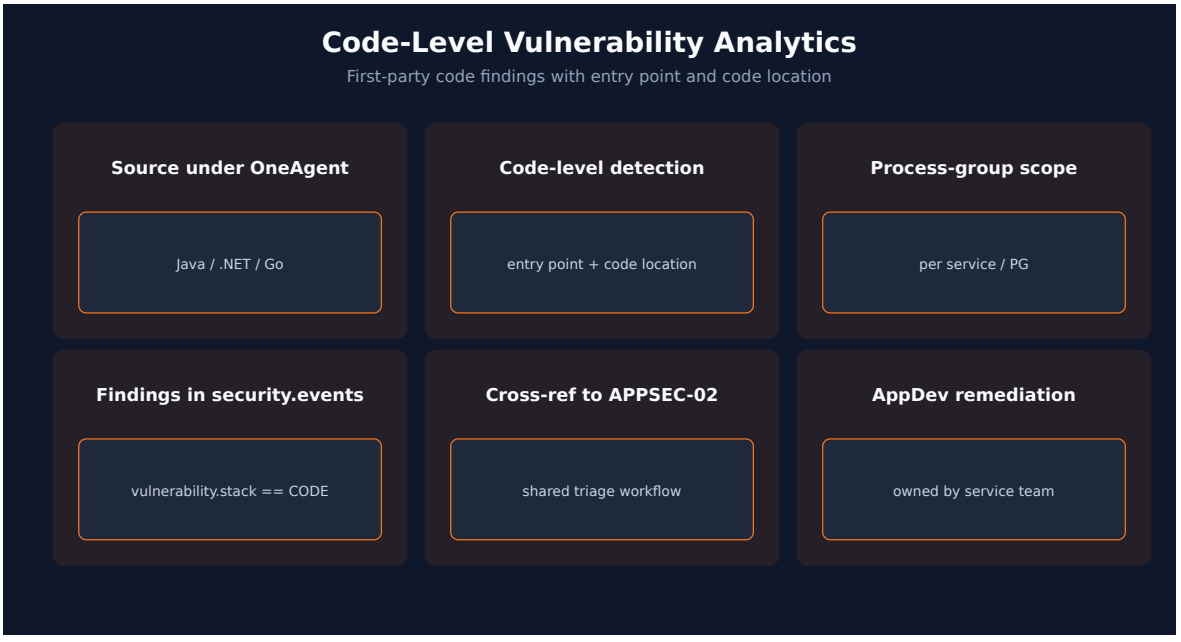


Table of Contents

1. 1. First-Party vs Third-Party
2. 2. Supported Runtimes
3. 3. Entry Point and Code Location
4. 4. DQL: Code-Level Findings
5. 5. Process-Group Scoping
6. 6. Next Steps
7. References

Prerequisites

Requirement	Details
Dynatrace Environment	Gen3 SaaS with Grail; AppSec entitlement enabled
OneAgent	Full-Stack mode (or code-module attached) on monitored hosts
Read access	To run the DQL: `storage:security.events:read` **plus** `storage:buckets:read` (a table permission alone reads nothing). The Vulnerabilities and Threats & Exploits apps have their own requirements — see APPSEC-09 for the full model
Background	APPSEC-01 (fundamentals + three-pillar framing)

1. First-Party vs Third-Party

Both kinds of findings land in the same Grail bucket (`security.events`) and the same Vulnerabilities app, but they're not the same thing:

Dimension	Third-party (APPSEC-02)	Code-level (this notebook)
What's vulnerable	An imported library (a CVE record)	A pattern in your own source code

Dimension	Third-party (APPSEC-02)	Code-level (this notebook)
Remediation	Upgrade / patch / replace dep	Code change + redeploy
Owner	Platform / dependency lead	Service / AppDev team
Volume	High (every CVE feed update)	Lower (per-class, per-pattern detections)
How it is found	Component loaded by a running process, matched against vulnerability feeds	User-input flow through the application analyzed at runtime
Score	DSS, lowered from CVSS by environmental context	Always 10 / Critical

In practice these belong on the same dashboard but in separate queues — confusing the two slows remediation because the wrong team gets paged.

> **Sources:** Vulnerability Analytics (DT docs) — third-party: *"Detects libraries and runtime components as they are loaded by processes."*; code-level: *"Analyzes how user input flows through the application."*; Vulnerabilities concepts (DT docs) — *"The score of a code-level vulnerability is always 10 and the risk always Critical"* (re-read 10/02/2026). **Derived:** the side-by-side comparison and team-ownership framing is a synthesis.

2. Supported Runtimes

Code-level analytics requires deep introspection into the running application, so it is documented for a narrower set of runtimes than third-party (library) detection.

Runtime	Code-level detection	Minimum OneAgent	Notes
Java 8 or higher	✓	1.259	Windows x86 and Linux x86 only
.NET	✓	1.289	.NET Framework 4.5, .NET Core 3.0 or higher, 64-bit processes only; deep monitoring must be enabled manually per host

Runtime	Code-level detection	Minimum OneAgent	Notes
Go	✓	1.311	Deep monitoring must be enabled manually per host

Node.js, Python and PHP have third-party (library) detection only — no code-level detection. **Ruby** has neither.

Code-level analytics is also opt-in, in three steps: enable *Code-level Vulnerability Analytics* in the Application Security settings; set the per-technology detection control (Monitor / Do not monitor), which custom rules per process group can override (§ 5); and enable *code-level vulnerability evaluation* for each technology under **Settings > Collect and capture > General monitoring settings > OneAgent features**. Restart the affected processes after each step — until a process restarts, it reports nothing. SSRF code-level detection on Java (OneAgent 1.309+) needs one more OneAgent feature, *Java SSRF code-level vulnerability and attack evaluation*.

The support matrix updates per OneAgent release — re-verify against the docs at rollout time for each runtime you care about.

> **Sources:** Runtime Vulnerability Analytics (DT docs) — the *Supported technologies* tables for third-party and code-level detection, *"Dynatrace detects code-level vulnerabilities in the following technologies."*, *"Only .NET Framework 4.5, .NET Core 3.0 or higher, and 64-bit processes are supported."*, and *"custom rules override the global detection control for the selected technology"* (re-read 10/02/2026).

3. Entry Point and Code Location

Each code-level finding shows where the attack can enter and where the vulnerable call is made:

- **Entry point** — the URL path, the untrusted input and the payloads through which an attacker can reach the vulnerable code.
- **Code location** — the class, method and line where the vulnerable function is called, for example `SQL injection at DatabaseManager.updateBio():82`.

That is enough for a developer to find the line. It does not identify a commit — map the code location to your repository yourself. (*Vulnerable functions* are a different, third-party concept: the optional Java feature from APPSEC-02 § 2.)

Two things to verify when you onboard code-level coverage:

1. **The code location resolves to your own classes** on a sample finding. If it points into framework code, routing by owning team is harder.
2. **Service ownership is mapped** in Dynatrace (via tags or naming convention) so the workflow in APPSEC-08 can route the ticket to the right AppDev team automatically.

> **Sources:** Vulnerabilities concepts (DT docs) — *"Code location Shows where the actual vulnerability is in the code (the location where the vulnerable function is called from). Example: SQL injection at DatabaseManager.updateBio():82"*; the same page lists *Entry point* (URL path, untrusted input, payloads) under code-level vulnerabilities and *Vulnerable functions* under third-party vulnerabilities (re-read 10/02/2026); Vulnerability Analytics (DT docs) — *"Filter by code-level vulnerability evaluation and enable the feature for the technologies you want to monitor. Select Save changes . Restart your processes."*

4. DQL: Code-Level Findings

Code-level findings are the same `VULNERABILITY_STATE_REPORT_EVENT` records as third-party ones, distinguished by `vulnerability.stack == "CODE"` (an `experimental` field; library findings carry `CODE_LIBRARY`). Do **not** filter on `vulnerability.type` for this — that field is a CWE-style classification, so a filter such as `vulnerability.type == "CODE_LEVEL"` is valid DQL that silently matches nothing.

The query uses the entity-level snapshot, keeps the latest record per vulnerability + affected entity, and counts what is still open.

```
// Code-level vulnerabilities currently open, by affected entity (latest
snapshot)
fetch security.events, from:-7d
| filter event.provider == "Dynatrace"
| filter event.category == "VULNERABILITY_MANAGEMENT"
| filter event.type == "VULNERABILITY_STATE_REPORT_EVENT"
| filter event.level == "ENTITY"
| filter vulnerability.stack == "CODE"
| dedup {vulnerability.display_id, affected_entity.id}, sort:{timestamp desc}
| filter vulnerability.resolution.status == "OPEN"
| filter vulnerability.mute.status == "NOT_MUTED"
| summarize open = count(), by:{affected_entity.name,
vulnerability.risk.level}
| sort open desc
```

> **Validation note:** validated for syntax and field names on 09/18/2026 and executes cleanly; the validation tenant has no RVA data, so it returned 0 rows there. Expect real counts only on a tenant with code-level vulnerability detection enabled.
>
> **Sources:** Vulnerability events (DT semantic dictionary) — `vulnerability.stack` examples `CODE ; CODE_LIBRARY ; SOFTWARE ; CONTAINER_ORCHESTRATION` (re-read 09/18/2026).
> **Dictionary:** `vulnerability.stack` (`experimental`) — *"Level of the vulnerable component in the technological stack."*; `vulnerability.type` (`stable`) — *"Classification of the vulnerability based on commonly accepted enums, such as CWE."*; `vulnerability.risk.level` , `vulnerability.resolution.status` , `vulnerability.mute.status` , `vulnerability.display_id` (all `stable`), read 09/18/2026.

5. Process-Group Scoping

Code-level coverage is enabled per process group (PG) in many tenants. This matters operationally:

- A PG with code-level disabled produces *zero* code-level findings — easily mistaken for "this service is secure."
- Rolling out code-level coverage incrementally (per service, per team) is safer than flipping it tenant-wide.

- Confirm code-level coverage status as a regular item in the AppSec governance review (APPSEC-10) — coverage drift is a known posture risk.

> **Sources:** Application Security (DT docs) for the process-group / code-module dependency framing.
>
> **Softened:** the *coverage as governance check* recipe is community practice.

6. Next Steps

1. Confirm code-level coverage on the runtimes you care about — all three enablement steps in § 2, with process restarts — and don't infer it from APPSEC-02's volume.
2. Check that a sample finding's code location resolves to your own classes.
3. Read **APPSEC-08** to wire findings to AppDev team ticket queues.
4. Read **APPSEC-09** so AppDev gets the narrow-scoped read access they need without seeing other teams' findings.

References

Source	Coverage
Application Security (DT docs)	Code-level scope reference
Runtime Vulnerability Analytics (DT docs)	Supported technologies for third-party and code-level detection
Vulnerability events (DT semantic dictionary)	`vulnerability.stack` and related fields
IAM policy statements reference (DT docs)	Permission tokens

> ⚠️ **DISCLAIMER**: This information was AI generated and is provided "as-is" without warranty. It was produced as an independent, community-driven project and **not supported by Dynatrace**. Always refer to official Dynatrace documentation for the most current information.